Ad Soyad: _____	**Tarih:** _____

Lab 13 – Hafta 13 – *OBYS Client/Server Mimarisi: QTcpServer, Length-Prefix, LOGIN+Token, Paylaşılan Öğrenci Deposu*

Lab Hakkında

Bu laboratuvar çalışmasında Bölüm 13'te öğrendiğiniz tüm parçaları (sunucu mimarisi, bağlantı yönetimi, kimlik doğrulama, paylaşılan durum, parola hash'leme) bir araya getirip minimal ama eksiksiz bir **OBYS Client/Server** çifti ayağa kaldıracaksınız. Sunucu `QTcpServer` ile dinler, her istemci bağlandığında length-prefix çerçeveli JSON komutları işler: `LOGIN`, `OGRENCI_LISTE`, `OGRENCI_EKLE`, `LOGOUT`. İstemci `QTcpSocket` ile bağlanıp önce LOGIN olur, sunucudan oturum jetonu alır, sonra öğrenci listesini sorgular.

Süre: 2 saat **Önkoşul:** Lab 12 (POSIX echo + Qt TCP istemci/sunucu + JSON dispatch) **Gereksinimler:** Qt 6 (Core, Network), CMake, `nc` veya istek çerçeveleyen Python betiği

Derleme Komutları

Qt projeleri için CMake:

```
cd Lab13_OBYS_Client_Server/server
cmake -B build -DCMAKE_PREFIX_PATH=/path/to/Qt6
cmake --build build
./build/obys_sunucu 9200

cd ../client
cmake -B build -DCMAKE_PREFIX_PATH=/path/to/Qt6
cmake --build build
./build/obys_istemci 127.0.0.1 9200 ogr001 sifre123
```

Lab başlangıç kodları: `out/LabExamples/Lab13_OBYS_Client_Server/` dizininde mevcut. Her dosyada `// GOREV-N` yorumlarını dolduracaksınız. `cerceveleme` yardımcıları (Lab 12 Çalışma 3'ten) hazır gelir; aynı protokolü yeniden kullanacaksınız.

Protokol Özeti

Tüm mesajlar 4 baytlık (big-endian) uzunluk başlığı + UTF-8 JSON payload formatındadır.

```
Istemci -> Sunucu:
  {"komut":"LOGIN","kullanici":"ogr001","parola_hash":"<sha256-hex>"}
  {"komut":"OGRENCI_LISTE","token":"<token>"}
  {"komut":"OGRENCI_EKLE","token":"<token>","numara":"OGR099","ad":"...","gano":3.10}
  {"komut":"LOGOUT","token":"<token>"}

Sunucu -> Istemci:
  {"durum":"LOGIN_OK","token":"tok_..."}
  {"durum":"LOGIN_FAIL","hata":"Kimlik dogrulanamadi"}
  {"durum":"OK","kayitlar":[{"numara":"...","ad":"...","gano":3.21},
      ...]}
  {"durum":"YETKISIZ","hata":"Once LOGIN gerekli"}
```

1 Görev 1: QTcpServer İskelet ve Bağlantı Kabul (25 dk)

Ön Bilgi

Bölüm 13 Ünite 1 Konu 1'de gördüğünüz `QTcpServer`, dinleyici soketi yönetir; `listen` çağrısı ile belirtilen porta bind+listen yapar. Yeni bir istemci bağlandığında `newConnection` sinyali tetiklenir; `nextPendingConnection` ile soketi alırsınız. Bu görevde sunucunun ayağa kalkma ve kabul etme iskeletini kuracaksınız; iş mantığını sonraki görevlerde ekleyeceksiniz.

Görev 1.1 – Sunucu Sınıfı İskeleti

`server/obys_sunucu.h` ve `server/obys_sunucu.cpp` dosyalarını aç.

1. Yapıcıda `server_ = new QTcpServer(this)` olarak yarat.
2. `server_ newConnection` sinyalini sınıfın `yeniBaglanti` slot'una bağla (yeni connect syntax ile).
3. `baslat(quint16 port)` fonksiyonunda `server_->listen(QHostAddress::Any, port)` çağır; hata durumunda `errorString` ile log atıp `false` dön.
4. `yeniBaglanti` slot'unda `while (server_->hasPendingConnections())` döngüsü kur; her istemciyi `nextPendingConnection` ile al, `istemciler_` hash'ine boş tampon ile ekle.

Görev 1.2 – Soket Sinyallerini Bağla

`yeniBaglanti` içinde, kabul edilen her soket için:

- `readyRead` sinyalini `veriHazir` slot'una bağla.
- `disconnected` sinyalini `baglantiKesildi` slot'una bağla.
- `veriHazir` ve `baglantiKesildi` fonksiyonlarını şimdilik boş bırak (sonraki görevlerde dolduracaksın); fakat fonksiyon imzaları derlenebilir olmalı.

Doğrulama

Sunucuyu `./obys_sunucu 9200` olarak çalıştır. Başka terminalde:

```
nc 127.0.0.1 9200
# Hicbir cevap gelmemeli ama baglanti acik kalmali; sunucu logunda
# "Yeni istemci: ..." satiri gormelisin.
Ctrl+C
# Sunucu logunda "Baglanti kesildi" gormelisin (Gorev 5'te).
```

Dikkat: CMake `Network` Bileşeni

`find_package(Qt6 COMPONENTS Core Network REQUIRED)` ve `target_link_libraries(... Qt6::Network)` olmazsa `QTcpServer` için linker "undefined reference" verir. CMake dosyanı baştan kontrol et.

Yaygın Hatalar

1. `server_->listen(...)` çağrısının dönüşünü kontrol etmemek; başka süreç aynı portu tutuyorsa sessizce çalışmaz.

2. `QTcpServer::newConnection` yerine yanlış sinyal seçmek (ör. yok olan `connectionAccepted`); derleme hatası verir.
3. Soket-bazlı sinyalleri (readyRead/disconnected) bağlamayı unutmak; veri gelse bile hiçbir şey olmaz.

2 Görev 2: Length-Prefix Çerçeveleme + JSON Dispatch (25 dk)

Ön Bilgi

Bölüm 12 Ünite 4 Konu 1'de gördüğünüz length-prefix protokolünü Lab 12 Çalışma 3'te uygulamıştın. Aynı `cerceveleme` kütüphanesini bu lab'da yeniden kullanacaksın: `cerceveSar(payload)` 4 baytlık big-endian uzunluk + payload üretir; `cerceveCoz(buffer)` akümülatör buffer'dan tam çerçeveleri çıkarır. Payload UTF-8 kodlu bir JSON metnidir; `QJsonDocument::fromJson` ile parse edilir.

Görev 2.1 – veriHazir Slot'u

`veriHazir` slot'unda:

1. `qobject_cast<QTcpSocket*>(sender())` ile gönderen soketi al; `nullptr` ise dön.
2. İstemcinin akümülatör buffer'ını `istemciler_` hash'inden al; `readAll` sonucunu buffer'a ekle.
3. `cerceveleme::cerceveCoz(buffer)` ile tam çerçeveleri çıkar (`try/catch` ile koru).
4. Her çerçeve payload'ı için `komutuIsle(istemci, payload)` çağır; dönen JSON cevabını `cerceveleme::cerceveSar` ile sarmala ve sokete yaz.
5. Çerçeveleme istisnası gelirse `disconnectFromHost` ile bağlantıyı kes.

Görev 2.2 – komutuIsle Dispatcher

`komutuIsle(QTcpSocket*, const QByteArray& payload)` fonksiyonunda:

1. Payload'ı `QJsonDocument::fromJson` ile parse et; `QJsonParseError` hatasında `{"durum":"HATA","hata":"gecersiz JSON"}` dön.
2. `obj.value("komut").toString()` ile komut adını al.
3. if/else if zinciri:
   - `"PING"` → `{"durum":"OK","mesaj":"PONG"}`
   - `"LOGIN"` → `komutLogin` (Görev 3'te yazılacak)
   - `"OGRENCI_LISTE"` → `komutOgrenciListe` (Görev 4)
   - `"OGRENCI_EKLE"` → `komutOgrenciEkle` (Bonus)
   - `"LOGOUT"` → oturumu sil; `{"durum":"OK"}`
   - bilinmeyen → `{"durum":"HATA","hata":"Bilinmeyen komut"}`

Doğrulama

Aşağıdaki Python betiğiyle PING gönder ve PONG cevabı al:

```
python3 -c "
import socket, struct
s = socket.socket(); s.connect(('127.0.0.1', 9200))
p = b'{\"komut\":\"PING\"}'
s.sendall(struct.pack('>I', len(p)) + p)
n = struct.unpack('>I', s.recv(4))[0]
print(s.recv(n).decode())
"
# Beklenen: {"durum":"OK","mesaj":"PONG"}
```

Dikkat: `cerceveCoz` While Döngüsü

`cerceveCoz` `while` ile birden fazla tam çerçeveyi tek tetiklemede çıkarabilmeli. `if` ile yazarsan 5 mesaj birden gelirse 4'ü kayıp gider.

Yaygın Hatalar

1. Tek global buffer kullanmak (her istemci için ayrı buffer şart).
2. Cevap yazarken `cerceveSar` ile sarmamak; istemci frame parser'ı çöker.
3. `QJsonDocument` parse hatasını kontrol etmemek; bozuk payload'da `undefined behavior`.

3 Görev 3: LOGIN – Salt + SHA-256 Parola, Oturum Jetonu (30 dk)

Ön Bilgi

Bölüm 13 Ünite 2 Konu 2 LOGIN akışını, Ünite 4 Konu 1 ise `QCryptographicHash` ile SHA-256 parola hash'lemesini anlatıyor. Düz parolayı sunucuya hiçbir zaman saklamayız; yalnızca `salt + parola` dizisinin SHA-256 hex çıktısını saklarız. İstemci LOGIN sırasında parolanın hex hash'ini gönderir; sunucu kayıtlı hash ile karşılaştırır. Eşleşme varsa rastgele bir oturum jetonu (`token`) üretip istemciye döner ve `oturumlar_[soket] = token` olarak kaydeder.

Görev 3.1 – Demo Kullanıcı Veritabanı

`ObysSunucu` yapıcısında basit bir kullanıcı tablosu yarat:

- Demo kullanıcı: `ogr001` / parola `sifre123`
- Sakla: `kullaniciSifreler_[kullanici] = sha256_hex(parola)`
- `QCryptographicHash::hash(QByteArray, Sha256)` kullan; `toHex` ile hex string al.

Görev 3.2 – komutLogin Handler

`komutLogin(QTcpSocket* istemci, const QJsonObject& obj)`:

1. `kullanici = obj.value("kullanici").toString()` ve `hash = obj.value("parola_hash").toString()` oku.
2. Tabloyu sorgula: `kullaniciSifreler_.value(kullanici) == hash`?
3. Eşleşme varsa: `uretToken` ile rastgele jeton üret (`QDateTime::currentMSecsSinceEpoch` + `QRandomGenerator::global()->generate()` kombinasyonu yeter), `oturumlar_[istemci] = token` kaydet, `{"durum":"LOGIN_OK","token":"..."}` dön.
4. Eşleşme yoksa: `{"durum":"LOGIN_FAIL","hata":"Kimlik dogrulanamadi"}` dön. **Hatada hangi alanın yanlış olduğunu söyleme** (kullanıcı sayma saldırısı önleme).

Doğrulama

Python ile (sha256 hex'i hesapla, JSON gönder):

```
python3 -c "
import socket, struct, hashlib, json
h = hashlib.sha256(b'sifre123').hexdigest()
s = socket.socket(); s.connect(('127.0.0.1', 9200))
p = json.dumps({'komut':'LOGIN','kullanici':'ogr001',
                'parola_hash': h}).encode()
s.sendall(struct.pack('>I', len(p)) + p)
n = struct.unpack('>I', s.recv(4))[0]
print(s.recv(n).decode())
"
# Beklenen: {"durum":"LOGIN_OK","token":"tok_..."}
```

Yanlış parola ile LOGIN_FAIL dönmeli.

Dikkat: Düz Parola Tutmak Yasak

Sunucu hafızasında veya dosyasında **asla** açık parola tutma. Yalnızca `sha256(salt + parola)` hex'i saklanır. Bu lab'da basitlik için salt'ı atlıyoruz; gerçek üretimde `bcrypt`, `scrypt` veya `argon2` gibi yavaş, saltlı KDF'leri tercih edersin.

Yaygın Hatalar

1. Hash karşılaştırmasında string normalizasyonunu unutmak (büyük/küçük harf, boşluk).
2. Token üretiminde `std::rand` gibi tahmin edilebilir kaynak kullanmak; `QRandomGenerator::global` tercih edilir.
3. LOGIN_FAIL'de hangi alanın yanlış olduğunu söylemek (kullanıcı varlığını sızdırır).

4 Görev 4: OGRENCI_LISTE – Yetki Kontrolü + Paylaşılan Erişim (25 dk)

Ön Bilgi

Bölüm 13 Ünite 3 Konu 2'de gördüğümüz `std::shared_mutex` bir okuyucu-yazar kilididir: birden fazla okuyucu thread aynı anda `std::shared_lock` ile veriye erişebilirken, yazar tek başına `std::unique_lock` ile kilidi alır. OBYS sunucumuzda öğrenci listesi sık okunup nadir yazıldığı için bu desen idealdir. Bu görevde `komutOgrenciListe` handler'ı yetki kontrolü yapacak ve paylaşılan vector'ü güvenle okuyacak.

Görev 4.1 – Demo Veri ve Kilit

`ObysSunucu` sınıfına ekle:

- `std::vector<Ogrenci> ogrenciler_` (struct `Ogrenci{numara, adSoyad, gano}`)
- `mutable std::shared_mutex ogrenciKilit_`
- Yapıcıda 4–5 demo öğrenci ekle (OGR001..OGR004).

Görev 4.2 – komutOgrenciListe Handler

`komutOgrenciListe(QTcpSocket* istemci, const QJsonObject& obj)`:

1. `token = obj.value("token").toString()` oku.
2. Yetki kontrolü: `oturumlar_.value(istemci) != token || token.isEmpty()` ise `{"durum":"YETKISIZ","hata":"Once LOGIN gerekli"}` dön.
3. `std::shared_lock kilit(ogrenciKilit_)` ile kilidi al (RAII ile sonunda otomatik bırakılır).
4. Vector üzerinde dolaş, her öğrenciyi `QJsonObject` olarak `QJsonArray`'e ekle.
5. Cevap: `{"durum":"OK","kayitlar":[...]}`

Doğrulama

Önce LOGIN ile token al, sonra:

```
# token degiskenini onceki LOGIN cevabindan kopyala
p = json.dumps({'komut':'OGRENCI_LISTE','token': token}).encode()
s.sendall(struct.pack('>I', len(p)) + p)
# Cevap: {"durum":"OK","kayitlar":[{"numara":"OGR001",...}, ...]}
```

Token vermeden veya yanlış token ile YETKISIZ alman gerekir.

Dikkat: Hash'in Mutable Olması

`std::shared_mutex`'i `const` fonksiyonlarda da kilitleyebilmek için üye değişken `mutable` olmalıdır. Yoksa `const` okuma fonksiyonunda `lock` çağrısı derleme hatası verir.

Yaygın Hatalar

1. Yetki kontrolünü atlamak; istemci LOGIN olmadan listeyi okuyabiliyor.
2. `std::lock_guard` ile kilit alıp `std::unique_lock` sanmak; okumalar serileşir, paralelliği kaybedersin.

3. JSON dizisi inşasında manuel string birleştirme; özel karakter (", \) escape'lemeyi unutursun. `QJsonDocument` kullan, manuel string yapma.

5 Görev 5: baglantiKesildi – Oturum ve Buffer Temizliği (15 dk)

Ön Bilgi

Bölüm 13 Ünite 1 Konu 2'de işlediğimiz bağlantı yaşam döngüsü şöyle biter: istemci `disconnectFromHost` çağırınca veya soket kapanınca sunucuda `disconnected` sinyali tetiklenir. Bu noktada sunucunun istemciyle ilgili tüm durumu (akümülatör buffer, oturum jetonu) temizlemesi ve soketi `deleteLater` ile silmesi gerekir. Aksi halde bellek sızıntısı ve "zombi oturum" kalır.

Görev 5 – baglantiKesildi Slot'u

`baglantiKesildi` fonksiyonunda:

1. `qobject_cast<QTcpSocket*>(sender())` ile soketi al; `nullptr` ise dön.
2. `istemciler_.remove(istemci)` ile akümülatörü temizle.
3. `oturumlar_.remove(istemci)` ile oturumu kapat.
4. Log: "Bağlantı kesildi: `<peerAddress>`"
5. `deleteLater` ile soketi sil (`delete` yerine; çünkü olay döngüsü içinden silinmemeli).

Dikkat: `deleteLater` Şart

`disconnected` slot'undan dönmeden önce soketi `delete` edersen Qt olay döngüsü hâlâ üzerinde işlem yapıyor olabilir; çökme veya kullanım-sonrası-serbest hatası olur. `deleteLater` olay kuyruğuna ekler ve güvenli zamanda siler.

Yaygın Hatalar

1. `istemciler_` hash'inden silmemek: sonraki broadcast'te kapalı sokete yazma uyarısı.
2. `oturumlar_` silmemek: jeton sızıntısı; aynı kullanıcı yeniden bağlanırsa karışıklık olur.
3. Soketi `delete` etmek (`deleteLater` yerine): çökme.

6 Görev 6 (Bonus): OGRENCI_EKLE – unique_lock ile Yazar (15 dk)

Bonus

Bu görev zorunlu değildir; tamamlayanlar OBYS sunucusunu okuma-yazma karışık erişim için tam fonksiyonel hâle getirir.

Görev 6 – komutOgrenciEkle Handler

`komutOgrenciEkle(QTcpSocket* istemci, const QJsonObject& obj)`:

1. Yetki kontrolü (Görev 4 ile aynı).
2. Payload'dan `numara`, `ad`, `gano` oku; eksik veya tip uyumsuzsa `{"durum":"HATA","hata":"Eksik alan"}` dön.
3. Girdi doğrulaması (Bölüm 13 Ünite 4 Konu 3): numara boş değil, gano $\in [0, 4]$.
4. `std::unique_lock kilit(ogrenciKilit_)` ile yazar kilidini al.
5. Aynı numara zaten varsa `{"durum":"HATA","hata":"Numara mevcut"}` dön.
6. Vector'e push_back, `{"durum":"OK"}` dön.

Doğrulama

Iki ardışık Python istemcisinden biri OGRENCI_EKLE atarken diğeri OGRENCI_LISTE çekerse, listede yeni öğrencinin görünmesi gerekir. Aynı numara ile ikinci ekleme HATA dönmelidir.

Yaygın Hatalar

1. Yazma sırasında `std::shared_lock` kullanmak: veri yarışı! Yazar `std::unique_lock` olmalı.
2. Aynı kilidi hem yetki kontrolü hem ekleme için tutarsa parite yanlış: oturum kontrolü kilide girmeden **önce** yapılır.
3. Girdi doğrulamasını atlamak: `gano:9999` kabul edilirse veri kalitesi bozulur.

7 İstemci Tarafı: Konsol İstemci (10 dk)

Ön Bilgi

İstemci kodu sana **tamamına yakın hazır verilir**. Görevin sadece istemcinin sunucuyla başarıyla konuşabildiğini doğrulamaktır. `client/main.cpp` dosyasında akış: `QTcpSocket` ile bağlan → LOGIN gönder ve token al → OGRENCI_LISTE gönder ve cevabı yazdır → `disconnectFromHost`.
Lab 12'den farklı olarak istemcide de length-prefix çerçeveleme yapılır; bu nedenle `cerceveleme` kütüphanesini istemci CMakeLists.txt'e eklemek zorundasın.

Görev İstemci – Uçtan Uca Test

1. İstemciyi derle ve `./obys_istemci 127.0.0.1 9200 ogr001 sifre123` ile çalıştır.
2. Beklenen çıktı:

```
[Istemci] 127.0.0.1:9200 baglaniliyor...
[Istemci] Baglandi.
[Istemci] LOGIN cevabi: {"durum":"LOGIN_OK","token":"tok_..."}
[Istemci] LISTE cevabi:
  {"durum":"OK","kayitlar":[
     {"numara":"OGR001","ad":"Ali Yilmaz","gano":3.21}, ...
  ]}
```

3. Yanlış parola ile çağırınca (`./obys_istemci ... ogr001 yanlis`) LOGIN_FAIL alman gerekir.

Değerlendirme

Görev	Puan
Görev 1 – QTcpServer iskelet ve bağlantı kabul	15
Görev 2 – Length-prefix çerçeveleme + JSON dispatch	20
Görev 3 – LOGIN ve oturum jetonu	20
Görev 4 – OGRENCI_LISTE ve shared_mutex	20
Görev 5 – baglantiKesildi temizliği	10
İstemci ile uçtan uca test (LOGIN + LISTE çalışıyor)	15
Görev 6 (Bonus) – OGRENCI_EKLE + unique_lock	10
Toplam	**100 (+10)**

Hata Ayıklama

```
# Hangi port kullaniliyor:
lsof -i :9200

# JSON cikti formatlama:
python3 -c "...gonder..." | python3 -m json.tool

# TCP trafigini izle:
sudo tcpdump -i lo port 9200 -X
```

Sık görülen hatalar:

- "LOGIN cevabi gelmiyor" → length-prefix çerçeveleme istemci tarafında da yapılıyor mu? 4 baytlık header ekledin mi?
- "YETKISIZ alıyorum" → token sözcüğü tam olarak eşleşiyor mu? Kopyalarken boşluk düştü mü?
- "Sunucu çöküyor" → `deleteLater` yerine `delete` mi kullandın?